Cross-Encoder Reranking for Retrieval-Augmented Generation

Hamza Abdelaziz

***Abstract*—Retrieval-Augmented Generation (RAG) systems improve factual grounding by retrieving external evidence before answer generation, but standard dense retrieval can rank passages using vector-space similarity that is too broad for multi-hop question answering. This paper studies whether adding a lightweight cross-encoder reranking stage after dense retrieval improves evidence selection and downstream answer quality without using graph construction or persistent inter-passage relations. The proposed pipeline retrieves a Top-25 candidate pool using dense retrieval, reranks candidates using a cross-encoder, and passes the final Top-5 passages to a generator. The system is evaluated on 100 HotpotQA samples against Standard RAG, Large-pool RAG, and LLM Listwise Reranking. The pure cross-encoder improved Recall@5 from 0.710 to 0.780, Precision@5 from 0.284 to 0.312, and MRR from 0.866 to 0.909, while adding 3,095.5 ms mean latency compared with 16,957.2 ms for LLM Listwise Reranking. However, generation improvements were limited: faithfulness increased marginally while answer relevance decreased. Approximate aggregate-level statistical tests indicate that only LLM Listwise Reranking achieved a significant Recall@5 improvement after Bonferroni correction. These results position cross-encoder reranking as an efficient middle-ground enhancement to standard RAG, while showing that future systems require evidence-chain-aware reranking to improve multi-hop generation quality.**

***Index Terms*—Retrieval-Augmented Generation, cross-encoder reranking, dense retrieval, HotpotQA, multi-hop question answering, RAG evaluation.**

I. Introduction

Retrieval-Augmented Generation (RAG) has become a common architecture for grounding large language model (LLM) outputs in externally retrieved evidence. Instead of relying only on knowledge stored inside model parameters, a RAG system retrieves relevant passages from an external corpus and provides them as context to the generator. This design is useful for knowledge-intensive natural language processing tasks, where the needed information may be too large, domain-specific, or changing over time to be stored reliably inside a pretrained model alone [1]–[3].

A standard RAG pipeline usually contains two main stages: retrieval and generation. During ingestion, documents are divided into smaller chunks and encoded as dense vectors using a bi-encoder embedding model. During inference, the user query is encoded into the same vector space, and approximate nearest-neighbor search selects the top-k most similar passages. These passages are inserted into the generation prompt, and the LLM produces an answer grounded in the retrieved context [1], [4]. This architecture is simple and scalable, which is why it is used a lot in open-domain question answering and applied LLM systems.

However, the ranking mechanism used in standard dense retrieval has a limitation. Bi-encoder retrievers encode the query and passage independently, then compare them using a dot product or cosine similarity score. This is efficient because passage embeddings can be precomputed, but it also means that the retriever does not directly model detailed token-level interactions between the query and candidate passage. Because of this, dense retrieval may retrieve passages that are related to the question in topic but still not directly useful for answering it. The problem becomes clearer in multi-hop question answering, where the correct answer often needs multiple complementary evidence passages and not just one similar passage [5]–[7].

This paper addresses this limitation by inserting a cross-encoder reranking stage between dense retrieval and generation. The proposed pipeline first retrieves a Top-25 candidate pool using dense retrieval. Then, each query–passage pair is scored jointly by a cross-encoder model, which attends over the query and passage together and produces a relevance score. The candidates are reranked according to these scores, and the final Top-5 passages are passed to the generator. The architecture does not construct a knowledge graph, does not store persistent inter-passage relations, and does not require entity linking or graph traversal.

The main research question is: Does adding a cross-encoder reranking stage to a dense RAG pipeline improve retrieval precision and downstream answer quality compared with standard dense retrieval baselines? The hypothesis is that cross-encoder reranking over a larger candidate pool will improve the quality of the final context window because the cross-encoder jointly models the interaction between the query and each candidate passage.

The contribution of this paper is threefold. First, it presents a lightweight two-stage retrieval-reranking architecture for RAG. Second, it evaluates the proposed architecture against Standard RAG, Large-pool RAG, and LLM Listwise Reranking on 100 HotpotQA samples. Third, it analyzes retrieval quality, generation quality, latency, and statistical significance to determine whether the observed improvements are meaningful under the experimental setup.

II. Literature Review

A. Foundations of Retrieval-Augmented Generation

The foundations of modern RAG are closely related to retrieval-augmented language modeling and dense passage retrieval. REALM introduced the idea of retrieving external documents during language model pretraining and question answering [2]. Lewis et al. later formalized Retrieval-Augmented Generation as a framework combining a neural retriever with a sequence-to-sequence generator for knowledge-intensive NLP tasks [1]. Dense Passage Retrieval (DPR) showed that dense bi-encoder retrieval could outperform traditional sparse retrieval methods such as BM25 on open-domain question answering tasks [4].

Later RAG variants improved how retrieved passages are used by the generator. Fusion-in-Decoder (FiD) processes multiple retrieved passages separately in the encoder and fuses them in the decoder, showing that passage ordering and evidence coverage influence generation quality [8]. Self-RAG adds self-reflection and retrieval-control mechanisms, allowing the model to decide when retrieved evidence is useful [9]. These works show that retrieval quality is central to RAG performance, but they also show that retrieval quality must be evaluated together with answer generation quality rather than as an isolated component.

B. Dense Retrieval and Its Limitations

Dense bi-encoder retrieval is efficient because document embeddings are computed offline and compared quickly at query time. However, this efficiency comes with a ranking limitation. Since the query and passage are encoded independently, the model cannot directly attend to the exact words and relationships shared between them during scoring [4], [10]. This can weaken precision, especially when the corpus contains distractor passages that are topically similar but not actually useful for answering the question.

Unsupervised and weakly supervised retrievers such as Contriever improved dense retrieval by learning more generalizable representations [11]. However, they remain bi-encoder systems and therefore still rely on independent representation comparison. For multi-hop question answering, this limitation is important because the answer often requires retrieving two or more related passages that collectively form an evidence chain. A passage may be individually relevant but incomplete, or it may be useful only when paired with another passage.

C. Reranking Methods

Reranking is a common solution to the precision limitations of first-stage retrieval. Instead of using an expensive model over the entire corpus, a fast retriever first produces a candidate pool, and a more precise model reranks only those candidates.

Cross-encoders are suitable for this second stage because they jointly encode the query and passage in one transformer input. This allows attention to operate across both texts, capturing lexical overlap, semantic alignment, and entailment-like signals that bi-encoders cannot directly model [12], [13].

The disadvantage of cross-encoders is computational cost. Since each query–passage pair must be processed separately, cross-encoders are too expensive for first-stage retrieval over a large corpus. But they are practical for reranking a limited candidate pool such as Top-25.

Another reranking strategy is LLM listwise reranking, where a language model is prompted to rank all candidate passages by usefulness. LLM listwise reranking may better approximate answer-aware evidence selection, but it is usually slower and more expensive than cross-encoder scoring.

D. Graph-Based Retrieval Augmentation

Graph-based RAG systems try to improve multi-hop reasoning by explicitly modeling relationships between passages, entities, or summaries. HippoRAG constructs memory-inspired retrieval graphs to support multi-hop retrieval [14]. GraphRAG builds graph structures over document collections to improve global sensemaking and relational retrieval [15]. RAPTOR recursively builds tree structures over document summaries, supporting retrieval at different levels of abstraction [16]. KG2RAG and related knowledge-graph-based approaches use graph expansion to retrieve evidence that is connected through entities or relations [17].

These methods are powerful because multi-hop reasoning is naturally relational. However, graph-based systems introduce additional infrastructure requirements, including graph construction, entity extraction, relation extraction, graph storage, traversal rules, and maintenance as the corpus changes. This paper focuses on a simpler non-graph design choice: whether a cross-encoder reranker can produce meaningful retrieval improvements while preserving the modularity of standard dense RAG.

E. Evaluation of RAG Systems

RAG evaluation is difficult because retrieval metrics and generation metrics measure different parts of the system. Recall and precision measure whether relevant passages appear in the retrieved context, while answer faithfulness and answer relevance measure whether the generated response is grounded and answers the question [18].

RAGAS provides automated metrics for faithfulness and answer relevancy, making it useful when human evaluation is not feasible [19]. ARES also proposes automated RAG evaluation using lightweight judges [20].

The literature therefore suggests that a complete RAG evaluation should report both retrieval-level and generation-level metrics. A reranker may improve retrieved context quality without improving generated answers if the generator still fails to synthesize the evidence. This distinction directly informs the methodology of the present study.

III. Methodology

A. Research Design

This paper follows an experimental study design. The experiment evaluates whether adding cross-encoder reranking after dense retrieval improves RAG performance compared

with dense-retrieval baselines and LLM-based reranking. The independent variable is the retrieval or reranking strategy used before generation. The dependent variables are retrieval quality, generation quality, latency, and throughput.

The research question is:

RQ1: Does cross-encoder reranking improve retrieval precision and downstream answer quality compared with standard dense retrieval baselines in a RAG pipeline?

The hypothesis is:

H1: Cross-encoder reranking over a Top-25 dense retrieval candidate pool will improve the relevance of the final Top-5 context window compared with Standard RAG and Large-pool RAG because the cross-encoder jointly models query–passage interactions.

B. System Overview

The proposed system follows a two-stage retrieval-reranking design. In the first stage, a dense retriever retrieves a candidate pool from a FAISS index. In the second stage, a cross-encoder reranker scores each query–passage pair and reorders the candidates. The final Top-5 passages are then passed to the generator.

The system uses a non-graph design. It does not build a knowledge graph, does not store explicit inter-passage edges, and does not perform graph traversal. This keeps the architecture close to standard RAG while adding only one modular reranking component.

C. Document Ingestion

Source documents are divided into chunks using a sentence-boundary-aware splitting process. The target chunk size is 512 tokens with an overlap of 64 tokens. Each chunk is stored with metadata including chunk identifier, document identifier, title, chunk index, and content.

Chunks are embedded using `all-mpnet-base-v2`, producing 768-dimensional embeddings. Where applicable, local embedding-based relevancy calculations may also use `bge-small-en-v1.5`. Chunk embeddings are stored in a FAISS flat L2 index to preserve reproducibility and avoid approximate-index randomness.

D. Query-Time Retrieval

At inference time, the input question is embedded using the same embedding model used during ingestion. The dense retriever searches the FAISS index and returns the Top-25 candidate passages. This candidate pool is larger than the final context window so that the reranker has enough candidates to reorder.

E. Cross-Encoder Reranking

Each of the 25 candidate passages is paired with the input query and passed to `cross-encoder/ms-marco-MiniLM-L-6-v2`. The model produces a scalar relevance score for each query–passage pair.

Let q denote the query and

$$C = \{c_1, c_2, \ldots, c_{25}\}$$

denote the candidate set. For each candidate c_i, the cross-encoder produces a score

$$s_i = f(q, c_i).$$

Candidates are sorted in descending order of s_i, and the top five are selected as the final context.

F. Hybrid Reranking

A hybrid reranking variant combines the dense retrieval score with the cross-encoder score. Scores are min-max normalized per query, then combined as:

$$s_{\text{hybrid}} = \alpha s_{\text{dense}} + (1 - \alpha) s_{\text{CE}},$$

where s_{dense} is the normalized dense similarity score and s_{CE} is the normalized cross-encoder score.

The experiment used a fixed exploratory value of

$$\alpha = 0.3.$$

The hybrid result should be interpreted as the performance of this fixed value rather than as a fully optimized hybrid configuration.

G. Answer Generation

The final Top-5 passages are inserted into a RAG prompt and passed to the generator. The main answer generation model is `nvidia/nemotron-3-super-120b-a12b:free`, accessed through OpenRouter.

The system also supports `openai/gpt-oss-120b:free` as a fallback generator. Generation temperature is set to 0.0 across all conditions to reduce randomness and make comparisons more controlled.

H. Baselines

The experiment compares the proposed cross-encoder configurations against three baselines:

1) **Standard RAG:** retrieves the Top-5 passages directly from dense retrieval without reranking.
2) **Large-pool RAG:** retrieves a larger Top-25 pool but still selects the final Top-5 using dense similarity without cross-encoder reranking.
3) **LLM Listwise Reranking:** retrieves the Top-25 candidate pool and uses an LLM-based listwise reranking prompt to order candidates before answer generation.

Graph-based RAG is not used as an experimental baseline in this study. The comparison to graph-based methods is kept in the literature review and discussion because it helps position the proposed architecture against structurally richer RAG systems, but the empirical comparison focuses on dense retrieval and reranking variants.

I. Dataset

The evaluation uses 100 HotpotQA samples in the distractor setting. HotpotQA is appropriate because it contains multi-hop questions requiring evidence from multiple passages.

This scope was chosen to keep the experiment controlled around one multi-hop benchmark while still allowing direct comparison between the evaluated retrieval and reranking configurations. MuSiQue is not part of this experimental scope.

J. Evaluation Metrics

Retrieval quality is evaluated using Recall@5, Recall@10, Precision@5, and Mean Reciprocal Rank (MRR).

Recall@5 measures the proportion of gold supporting evidence retrieved in the final Top-5 context window. Precision@5 measures the proportion of retrieved Top-5 passages that are gold supporting evidence. MRR measures the reciprocal rank of the first relevant passage.

Recall@10 is included for transparency in the results table. In this implementation, the final evaluated context is Top-5, so Recall@10 is not independently informative. A separate Recall@10-focused evaluation would need to store and evaluate ten retrieved passages per query.

Generation quality is evaluated using RAGAS answer faithfulness and answer relevancy. The RAGAS judge model is `openai/gpt-4o-mini`, accessed through OpenRouter. If the RAGAS model parameter is not specified, the system falls back first to the configured generation model and then to the hard-coded default of GPT-4o mini. Embedding-based relevancy calculations use local embedding models and are not accessed through OpenRouter.

Efficiency is evaluated using mean latency in milliseconds and throughput in queries per second (QPS).

K. Statistical Testing

Statistical testing is conducted using approximate aggregate-level two-proportion z-tests with Bonferroni correction, Wilson score confidence intervals, bootstrap confidence intervals where per-query data are available, and Cohen's h effect size estimates.

Because per-query results were not available for all systems, fully paired non-parametric tests such as the Wilcoxon signed-rank test could not be applied across all conditions. The reported statistical tests are therefore treated as approximate evidence rather than definitive paired significance tests.

L. Hyperparameter Configuration

- Chunk size: 512 tokens
- Chunk overlap: 64 tokens
- Retrieval pool size: Top-25
- Final context window: Top-5
- Cross-encoder model: `cross-encoder/ms-marco-MiniLM-L6-v2`
- Primary embedding model: `all-mpnet-base-v2`
- Additional local embedding model: `bge-small-en-v1.5`
- Hybrid weight: Fixed at $\alpha = 0.3$, exploratory; not tuned
- Main generator: `nvidia/nemotron-3-super-120b-a12b:free`
- Fallback generator: `openai/gpt-oss-120b:free`
- RAGAS judge model: `openai/gpt-4o-mini` through OpenRouter
- Generation temperature: 0.0

M. Ethical Considerations

This study uses publicly available benchmark data and does not collect private, sensitive, or personally identifiable data. No human participants are involved. Generated answers are evaluated only for research purposes and are not used for real-world decisions.

IV. Results

This section reports the experimental results obtained from the evaluation run on 100 HotpotQA questions. All evaluated configurations used the same dataset subset, the same corpus index, the same final context window size of Top-5, and the same generation settings. The evaluated systems include three baselines and two proposed cross-encoder configurations.

The empirical scope is intentionally focused on HotpotQA because it gives a direct multi-hop QA setting where retrieved evidence quality matters. Graph-based RAG is discussed as related work and as a comparison point in terms of architecture, but the measured experiments compare dense retrieval, large-pool retrieval, LLM listwise reranking, and cross-encoder reranking.

A. Overall Quantitative Results

Table I reports the full results across all evaluated configurations.

The strongest overall performance was achieved by LLM Listwise Reranking. It reached the highest Recall@5 and Recall@10 (0.885), Precision@5 (0.354), MRR (0.957), faithfulness (0.766), and answer relevance (0.661). It also had the highest mean latency at 26,821.9 ms and the lowest throughput at 0.037 QPS.

The pure cross-encoder reranker improved retrieval performance over Standard RAG. Recall@5 increased from 0.710 to 0.780, Precision@5 increased from 0.284 to 0.312, and MRR increased from 0.866 to 0.909. So the reranker did improve the ordering of evidence passages inside the final Top-5 context window.

However, the retrieval improvement did not consistently improve generation quality. Faithfulness increased only slightly from 0.594 to 0.597, while answer relevance decreased from 0.563 to 0.538. This is not the ideal outcome, but it shows a real difference between improving retrieved passages and improving the final answer.

The hybrid cross-encoder configuration also improved retrieval over Standard RAG, with Recall@5 of 0.775, Precision@5 of 0.310, and MRR of 0.892. However, it underperformed the pure cross-encoder on retrieval metrics and reduced faithfulness to 0.562. Since $\alpha = 0.3$ was fixed, this result mainly reflects this specific hybrid setting.

Large-pool RAG produced no retrieval improvement over Standard RAG. It matched Standard RAG exactly on Recall@5, Recall@10, Precision@5, and MRR, while increasing

TABLE I
OVERALL RESULTS ACROSS RAG CONFIGURATIONS

Setup	Group	R@5	R@10	P@5	MRR	Faith.	Relev.	Lat. (ms)	QPS
Standard RAG	Baseline	0.710	0.710	0.284	0.866	0.594	0.563	9,864.7	0.101
Large-pool RAG	Baseline	0.710	0.710	0.284	0.866	0.587	0.549	26,713.1	0.037
LLM Listwise Reranking	Baseline	0.885	0.885	0.354	0.957	0.766	0.661	26,821.9	0.037
CE Rerank, pure	Proposed	0.780	0.780	0.312	0.909	0.597	0.538	12,960.2	0.077
CE Hybrid, $\alpha = 0.3$	Proposed	0.775	0.775	0.310	0.892	0.562	0.562	13,662.1	0.073

latency substantially. This means that retrieving a larger candidate pool by itself is not useful unless the system has an effective reranking mechanism.

B. Differences Relative to Standard RAG

Table II reports the absolute differences between each condition and Standard RAG.

The pure cross-encoder produced a Recall@5 improvement of 0.070, a Precision@5 improvement of 0.028, and an MRR improvement of 0.043 over Standard RAG. Its mean latency increased by 3,095.5 ms.

LLM Listwise Reranking produced the largest retrieval and generation improvements but added 16,957.2 ms of latency. So the cross-encoder result sits between Standard RAG and LLM Listwise Reranking: not the strongest result, but a better latency-quality trade-off than LLM-based reranking.

C. Recall@10 Observation

Recall@10 is identical to Recall@5 across all evaluated conditions. This does not mean that retrieving ten passages is always useless. It happens because the evaluated final context is Top-5, so Recall@10 is not independently informative in this run. It is still reported because it belongs to the metric list, but it should not be overinterpreted.

D. Latency and Throughput

The latency results show a clear trade-off between quality and cost. Standard RAG was the fastest condition with a mean latency of 9,864.7 ms and throughput of 0.101 QPS. The pure cross-encoder increased mean latency to 12,960.2 ms while preserving 0.077 QPS. The hybrid cross-encoder was slightly slower at 13,662.1 ms and 0.073 QPS.

Large-pool RAG and LLM Listwise Reranking had nearly identical mean latencies: 26,713.1 ms and 26,821.9 ms respectively. The difference between them is only 108.8 ms, even though their retrieval logic is different. This makes sense because both conditions involve a larger retrieval pool and a heavier workflow than Standard RAG. The cross-encoder configurations are much more efficient than both large-pool and LLM listwise conditions.

E. Statistical Testing

Table III reports Wilson 95% confidence intervals for Recall@5 and a bootstrap confidence interval for MRR where available.

Table IV reports two-proportion z-tests comparing each condition against Standard RAG on Recall@5 and Precision@5. Bonferroni correction was applied for four pairwise comparisons, producing a corrected threshold of $\alpha = 0.0125$.

Only LLM Listwise Reranking achieved a statistically significant improvement over Standard RAG after Bonferroni correction, specifically on Recall@5. Cross-encoder reranking improved the observed retrieval metrics, but the improvements did not reach statistical significance under the corrected threshold. This means the cross-encoder result is promising in the measured values, but the statistical evidence is not strong enough to claim a confirmed improvement.

Table V reports Cohen's h effect sizes for Recall@5 differences compared with Standard RAG.

The effect-size results are consistent with the significance tests. The cross-encoder effects are positive but small under the current evaluation. Because the experiment includes 100 questions, the study can miss smaller effects. A larger evaluation would be useful, but the current sample still gives a controlled comparison of the systems.

V. DISCUSSION

The results partially support the research hypothesis. The pure cross-encoder reranker improved observed retrieval quality compared with Standard RAG, increasing Recall@5 from 0.710 to 0.780 and MRR from 0.866 to 0.909. This supports the idea behind the architecture: a cross-encoder can improve evidence ordering by jointly modeling the query and passage instead of relying only on independent dense-vector similarity.

At the same time, the results show that better retrieval metrics do not automatically lead to better generated answers. The pure cross-encoder improved retrieval, but faithfulness increased by only 0.003 and answer relevance decreased by 0.025. This means that passage-level relevance and answer-level quality are connected, but they are not the same thing. A reranker may select passages that are individually relevant to the query, while the generator may still fail to combine them into a complete and direct multi-hop answer.

This finding fits the broader RAG evaluation literature, which argues that retrieval metrics alone are not enough for evaluating RAG systems [18]–[20]. In HotpotQA-style questions, the answer often requires combining multiple evidence passages. If the retrieved context contains one strong passage but misses the complementary passage needed for reasoning, the generated answer can still be incomplete. Also,

TABLE II
ABSOLUTE DIFFERENCES COMPARED WITH STANDARD RAG

Setup	ΔR@5	ΔR@10	ΔP@5	ΔMRR	ΔFaith.	ΔRelev.	ΔLat. (ms)
Large-pool RAG	0.000	0.000	0.000	0.000	-0.007	-0.014	+16,848.4
LLM Listwise Reranking	+0.175	+0.175	+0.070	+0.091	+0.172	+0.098	+16,957.2
CE Rerank, pure	+0.070	+0.070	+0.028	+0.043	+0.003	-0.025	+3,095.5
CE Hybrid, $\alpha = 0.3$	+0.065	+0.065	+0.026	+0.026	-0.032	-0.001	+3,797.4

TABLE III
WILSON 95% CONFIDENCE INTERVALS FOR RECALL@5 AND BOOTSTRAP 95% CI FOR MRR

!

Configuration	R@5	95% CI (R@5)	MRR	95% CI (MRR)
Standard RAG	0.710	[0.615, 0.790]	0.866	[0.810, 0.917]
Large-pool RAG	0.710	[0.615, 0.790]	–	–
LLM Listwise	0.885	[0.808, 0.934]	–	–
CE Rerank, pure	0.780	[0.689, 0.850]	–	–
CE Hybrid, $\alpha = 0.3$	0.775	[0.684, 0.846]	–	–

if relevant passages are present but surrounded by distractors, the generator may not use them correctly.

A likely explanation is a mismatch between the cross-encoder objective and the generation objective. The cross-encoder used in this study is trained to estimate query–passage relevance. RAGAS answer relevance, however, evaluates whether the final generated answer directly addresses the question. A passage can be relevant without being sufficient for answer synthesis. So the bottleneck is not only whether a passage is relevant, but whether the selected set of passages forms a complete evidence chain.

The LLM Listwise Reranking baseline achieved the strongest overall results. It improved Recall@5 by 0.175, Precision@5 by 0.070, MRR by 0.091, faithfulness by 0.172, and answer relevance by 0.098 compared with Standard RAG. It was also the only condition with a statistically significant Recall@5 improvement after Bonferroni correction. This suggests that the LLM reranker is more answer-aware than the cross-encoder because it evaluates candidates in a listwise context and may reason about which passages collectively support the answer.

However, LLM Listwise Reranking was also the most expensive configuration. Its mean latency was 26,821.9 ms, compared with 12,960.2 ms for the pure cross-encoder and 9,864.7 ms for Standard RAG. This makes LLM listwise reranking less attractive for systems where latency or cost matters. The pure cross-encoder is more of a practical middle ground: it improves retrieval quality more than Standard RAG while staying much faster than LLM-based reranking.

The Large-pool RAG baseline gives another useful result. It matched Standard RAG on all retrieval metrics while substantially increasing latency. This shows that simply retrieving a larger pool does not improve performance unless the system has a stronger mechanism for selecting the best final context. The candidate pool is only useful if reranking can actually use it.

The hybrid cross-encoder configuration did not improve over the pure cross-encoder. It slightly reduced retrieval quality and lowered faithfulness. This does not prove that hybrid scoring is generally bad. It only shows that the fixed $\alpha = 0.3$ setting was not better in this run. Another value could behave differently.

The statistical results limit how strong the conclusion can be. Although the cross-encoder improved the observed metrics, the gains were not statistically significant after correction. This does not mean the method has no effect; it means the evidence from this setup is not strong enough to prove the effect statistically. The confidence intervals overlap, the effect sizes are small, and the tests are approximate because fully paired per-query data were not available for all systems.

Compared with graph-based RAG systems, the proposed architecture is simpler and easier to deploy. Graph-based systems are likely to perform better on complex multi-hop reasoning because they explicitly model relationships between entities, passages, and summaries. However, they require additional components such as graph construction, entity linking, graph storage, and traversal logic. The proposed cross-encoder architecture does not provide this relational reasoning layer, but it is modular and compatible with existing dense RAG systems.

The best interpretation of the results is not that cross-encoder reranking is superior to advanced RAG methods. Instead, cross-encoder reranking is useful when the goal is to improve evidence ranking without adding the full complexity of graph-based retrieval or the full latency of LLM-based reranking. It is a practical enhancement to Standard RAG, but it is not a complete solution to multi-hop answer generation.

VI. CONCLUSION

This study investigated whether cross-encoder reranking can improve a dense RAG pipeline for multi-hop question answering while avoiding the complexity of graph-based retrieval augmentation and the high latency of LLM-based reranking. The proposed architecture retrieves a Top-25 dense candidate pool, reranks it using a cross-encoder, and passes the final Top-5 passages to the generator.

The answer to the research question is mixed. Cross-encoder reranking improved observed retrieval quality compared with Standard RAG: Recall@5 increased from 0.710 to 0.780, Precision@5 increased from 0.284 to 0.312, and MRR increased from 0.866 to 0.909. This supports the hypothesis that joint query–passage scoring can improve evidence ordering compared with dense similarity alone. However, the improvement

TABLE IV
TWO-PROPORTION Z-TESTS VS STANDARD RAG WITH BONFERRONI CORRECTION

Comparison	Metric	z	p-value	Uncorrected	Bonferroni
Large-pool vs Standard	Recall@5	0.000	1.0000	n.s.	n.s.
LLM Listwise vs Standard	Recall@5	+3.079	0.0021	**	*
CE pure vs Standard	Recall@5	+1.136	0.2561	n.s.	n.s.
CE hybrid vs Standard	Recall@5	+1.051	0.2932	n.s.	n.s.
Large-pool vs Standard	Precision@5	0.000	1.0000	n.s.	n.s.
LLM Listwise vs Standard	Precision@5	+2.375	0.0176	*	n.s.
CE pure vs Standard	Precision@5	+0.968	0.3331	n.s.	n.s.
CE hybrid vs Standard	Precision@5	+0.900	0.3683	n.s.	n.s.

TABLE V
COHEN'S h EFFECT SIZE FOR RECALL@5 DIFFERENCES VS STANDARD RAG

Comparison	Cohen's h	Interpretation
Large-pool RAG vs Standard RAG	+0.000	Negligible
LLM Listwise vs Standard RAG	+0.445	Small
CE Rerank, pure vs Standard RAG	+0.161	Negligible
CE Hybrid, $\alpha = 0.3$ vs Standard RAG	+0.149	Negligible

did not reach statistical significance under the 100-sample evaluation, and it did not consistently improve generation quality.

The strongest overall method was LLM Listwise Reranking, which achieved the best retrieval and generation metrics and was the only condition with a statistically significant Recall@5 improvement after Bonferroni correction. However, it also introduced the highest latency. Cross-encoder reranking therefore represents a practical compromise: it is less powerful than LLM listwise reranking but substantially more efficient and easier to integrate into a standard RAG pipeline.

The study has several limitations. The evaluation focuses on 100 HotpotQA samples, so the findings should be read within that scope. The empirical baselines focus on dense retrieval, large-pool retrieval, LLM listwise reranking, and cross-encoder reranking, while graph-based systems are used only as a conceptual comparison. Recall@10 is not independently informative because the evaluated final context is Top-5. Statistical testing was also limited by incomplete per-query data across all systems. Finally, the hybrid weight $\alpha = 0.3$ was fixed rather than tuned.

Future work should evaluate the system on larger HotpotQA samples and on MuSiQue to test whether the same patterns hold under harder compositional reasoning. It should also include a graph-based baseline to compare lightweight reranking against relational retrieval empirically. Most importantly, future work should develop evidence-chain-aware reranking methods that rank sets of passages by whether they collectively support the reasoning path required by the question, rather than ranking passages only by individual relevance.

Overall, cross-encoder reranking is a promising lightweight enhancement for RAG retrieval quality, but the results show that retrieval precision alone is not enough. Multi-hop RAG systems also need better evidence-chain selection and answer synthesis mechanisms to convert improved retrieval into consistently better generated answers.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in *International Conference on Machine Learning*, 2020.

[3] S. Borgeaud et al., "Improving Language Models by Retrieving from Trillions of Tokens," in *International Conference on Machine Learning*, 2022.

[4] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of EMNLP*, 2020.

[5] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in *Proceedings of EMNLP*, 2018.

[6] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop Questions via Single-hop Question Composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[7] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of SIGIR*, 2020.

[8] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in *Proceedings of EACL*, 2021.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *International Conference on Learning Representations*, 2024.

[10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[11] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised Dense Information Retrieval with Contrastive Learning," *Transactions on Machine Learning Research*, 2022.

[12] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.

[13] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of ACL*, 2023.

[14] Y. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," arXiv preprint, 2024.

[15] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv preprint arXiv:2404.16130, 2024.

[16] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," in *International Conference on Learning Representations*, 2024.

[17] Z. Zhu, X. Yuan, H. Wang, J. Li, and W. Hu, "Knowledge Graph-Guided Retrieval Augmented Generation," arXiv preprint, 2024.

[18] A. Salemi and H. Zamani, "Evaluating Retrieval Quality in Retrieval-Augmented Generation," in *Proceedings of SIGIR*, 2024.

[19] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," arXiv preprint arXiv:2309.15217, 2023.

[20] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of NAACL*, 2024.

[21] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension," in *Proceedings of ACL*, 2017.

[22] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories," in *Proceedings of ACL*, 2023.